

AdaptiveTile: Content-Aware Parallel Image Processing with Predictive Tile Scheduling

HUMAIRA AYESHA*, Department of Computer Science and Engineering, Bangladesh

Image processing based on tiles maps naturally to multicore CPUs and GPUs, but in practice, the assumption that the cost of a single tile of computation is uniform is not practical. Tiles that are rich in texture or have sharp edges need a lot more computation than smooth regions, and ignoring this heterogeneity produces load imbalance and lower parallel efficiency. We introduce a system that combines content-aware feature extraction with machine learning (ML)-based complexity prediction, to guide scheduling in multiple execution modes: CPU serial, CPU parallel (dynamic and predictive scheduling), and GPU execution using CUDA streams. We train 576 tiles of 24 Kodak benchmark images, each extracting five texture and gradient-based features per tile, and test eight tabular regressors and three convolutional neural network (CNN) regressors to predict processing time per tile. Random Forest gives the optimal results once statistically informed preprocessing, with Spearman correlation of 0.584 and Mean Absolute Error (MAE) of 0.0153 ms. The highest average speedup of 4.76 $\times$ over serial execution of CPU-computable programs using six workers, and up to 8.0 $\times$ speedup with stream-level parallelism on a GPU. Experiments of three publicly available datasets (Kodak, DIV2K and BSDS500) demonstrate that tile size of 128 $\times$ 128 pixels with dynamic scheduling maximize throughput whereas larger tiles (e.g. 512 $\times$ 512 pixels) significantly lower parallel efficiency. These findings can be useful in designing high-throughput, content-aware scheduling policies on heterogeneous hardware to process images.

CCS Concepts: • **Computer systems organization** $\rightarrow$ **Parallel architectures**; • **Computing methodologies** $\rightarrow$ *Machine learning*; *Image processing*.

Additional Key Words and Phrases: parallel computing, tile-based image processing, predictive scheduling, work stealing, load balancing, machine learning, GPU, CUDA

ACM Reference Format:

Humaira Ayesha. 2026. AdaptiveTile: Content-Aware Parallel Image Processing with Predictive Tile Scheduling. In *Proceedings of International Conference on Parallel Processing (ICPP '26)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The throughput requirements of modern image processing workloads, such as real-time medical imaging and large-scale satellite imaging, cannot be satisfied by single-threaded CPU pipelines. A common solution is tile-based decomposition: partitioning an image into rectangular, independently processable regions that can be distributed across parallel workers.

However, tiles are not computationally uniform. Operations such as Non-Local Means (NLM) denoising or morphological filtering require significantly more computation than smooth, featureless

*Student ID: 23301683

Author's Contact Information: Humaira Ayesha, Department of Computer Science and Engineering, Bangladesh, humaira.ayesha@g.bracu.ac.bd.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICPP '26, Online

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2025/08

<https://doi.org/XXXXXXX.XXXXXXX>

regions. If a scheduler ignores this variability, slow tiles accumulate on a subset of workers while others remain idle, leading to reduced parallel efficiency.

Problem. How can a parallel scheduler minimize total makespan while remaining computationally lightweight? More precisely, given image tiles with content-dependent processing times, the goal is to design an efficient load-balancing scheduler that introduces minimal overhead.

Contributions. This paper makes the following contributions:

- (1) **AdaptiveTile framework.** A complete pipeline including image ingestion, tiling, feature extraction, machine learning-based complexity prediction, scheduling, parallel execution on CPU and GPU, and tile merging.
- (2) **Comparative complexity prediction study.** A systematic evaluation of eight tabular machine learning models and three CNN-based regressors—SqueezeNet, MobileNetV3, and EfficientNet-B0—for predicting per-tile runtime using a statistically driven preprocessing pipeline.
- (3) **Four-scheduler benchmark.** Characterization of static, dynamic, predictive, and hybrid schedulers across heterogeneous pipelines, varying worker counts, tile sizes, and halo widths.
- (4) **CPU–GPU comparative analysis.** Measurement of speedup, parallel efficiency, load imbalance ratio, Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index Measure (SSIM) across CPU and GPU execution modes.
- (5) **Generalisation across datasets.** Testing on three datasets of different resolutions, content types and annotation characteristics.

The rest of the paper is structured in the following way. Section 2 reviews related work. The system architecture is discussed in Section 3. The methodology is discussed in Section 2. In Section 5 we present the datasets and exploratory analysis. In Section 6, the results of the experiment are presented. Section 8 measures complexity predictors. Section 9 discusses key findings. Lastly, the paper will be concluded in Section 6.5, which also provides the future directions of the paper.

2 Related Work

2.0.1 Parallel and GPU Image Processing. Image processing that uses a graphics processing unit has been widely researched. Ruan et al. [17] also established a solid baseline in the application of parallelism on GPUs in this field with significant speedups in image convolution and filtering tasks with CUDA. Another strategy that was proposed by Halide and which was earlier suggested, is complementary, not substituteive, of our runtime prediction strategy.

2.1 Tile-Based Decomposition

Scalable image processing is based on tile-based decomposition. Murray et al. [12] used dataflow tiling in Naiad to improve the performance of distributed systems. In addition to throughput, tiling is necessary when images are too big to fit into memory or when task-level parallelity is required that is finer than the granularity of hardware resources. An important design parameter is the halo (or ghost zone): a border of overlapping pixels repeated across multiple tiles to make sure that neighborhood-sensitive filters can be applied correctly at tile boundaries without introducing seam artifacts.

2.2 Work Stealing and Dynamic Scheduling

Blumofe and Leiserson [1] established the theoretical foundation of dynamic task scheduling, showing that randomized work-stealing achieves near-optimal expected makespan for multithreaded computations. This is complemented by the Longest Processing Time (LPT) heuristic proposed by Graham, which schedules tasks by assigning the longest ones first to minimize the critical

path length. Our predictive scheduler adapts LPT to the image-tile setting by replacing unknown processing times with machine learning-based estimates.

2.3 Predictive Scheduling

Predictive scheduling is widely studied in High-Performance Computing (HPC), where regression models estimate job runtimes based on historical metadata. We adapt this concept to fine-grained image tile execution, where runtimes are on the order of milliseconds or less. In this regime, predictor inference latency must be negligible relative to tile execution time, which constrains the choice of models to lightweight yet accurate predictors.

2.4 Literature Review Summary

Table 1 summarises related work and where AdaptiveTile differs.

Table 1. Literature Review Summary. Dynamic = dynamic work queue; LPT = Longest-Processing-Time heuristic; ML = machine learning predictor.

Reference	Year	Domain	Parallel	Scheduler	Predictor	Limitation
Graham [6]	1969	General	Multiprocessor	LPT (oracle)	None	Requires known times
Blumofe & Leiserson [1]	1999	General	Work-stealing	Dynamic	None	No domain-specific opt.
Canny [4]	1986	Vision	Serial	—	—	No parallelism
Ragan-Kelley [16]	2013	Image proc.	Multithreaded	Auto-tuned	None	No run-time prediction
Ruan et al. [17]	2019	Image proc.	GPU (CUDA)	Serial GPU	None	No CPU-GPU comparison
Breiman [2]	2001	ML	—	—	RF	Not applied to scheduling
AdaptiveTile (ours)	2026	Image proc.	CPU + GPU	Hybrid	RF/XGB	—

3 System Architecture

Figure 1 shows the AdaptiveTile system architecture. An input image moves through four logical stages: (1) tiling with halo borders, (2) feature extraction and complexity prediction, (3) scheduler assignment, and (4) parallel execution followed by output merging.

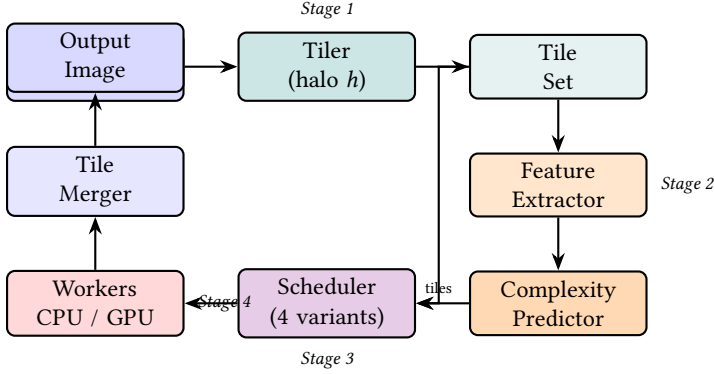


Fig. 1. AdaptiveTile system architecture. Stage 1 decomposes the image into halo-padded tiles. Stage 2 extracts content features and predicts tile processing time. Stage 3 schedules tiles to workers. Stage 4 executes tiles in parallel and reconstructs the output image.

4 Methodology

4.1 Tiling with Halo Borders

Given an image $I \in \mathbb{R}^{H \times W \times C}$ and tile size s , we partition I into $n_r \times n_c$ tiles where $n_r = \lceil H/s \rceil$ and $n_c = \lceil W/s \rceil$. Tile $T_{r,c}$ covers pixels $[rs, \min((r+1)s, H)) \times [cs, \min((c+1)s, W))$, extended on each side by a halo of width h pixels, clamped to image boundaries.

4.2 Feature Extraction

In order to describe the computational complexity of each tile, we compute seven features:

- (1) **Edge density:** Relative number of Canny edge pixels (thresholds: $T_{\text{low}} = 50$, $T_{\text{high}} = 150$).
- (2) **Gradient variance:** variance of Sobel gradient magnitude map.
- (3) **Intensity variance:** Variance of the intensity of grayscale pixel intensities.
- (4) **Entropy:** entropy of a 16-bin histogram of intensity.
- (5) Mean LBP with uniform patterns ($P = 8$, $R = 1$) [13].
- (6) Tile column: Column index encoding space.
- (7) Tile row: Row index represents spatial location.

Features 1-5 encode visual complexity; features 6-7 encode spatial priors (sky areas, etc. will tend to appear towards the top of outdoor images, and are less expensive to compute than textured foreground regions).

4.3 Complexity Predictor

The complexity predictor maps feature vectors $\mathbf{x} \in \mathbb{R}^7$ to predicted processing times $\hat{y} \in \mathbb{R}_{>0}$ (in milliseconds). We evaluate two families of models:

Tabular models: Linear Regression, Ridge Regression ($\alpha = 1$), Support Vector Regression (SVR) with an RBF kernel ($C = 10$), Random Forest (100 trees), Gradient Boosting (100 estimators), XGBoost [5], LightGBM [9], and a Multilayer Perceptron (MLP, hidden layers [64, 32]).

CNN models: SqueezeNet [8], MobileNetV3-Small [7], and EfficientNet-B0 [18], each fine-tuned as a single-output regressor (30 epochs, Adam optimiser, learning rate 10^{-4} , MSE loss). CNNs receive tile images resized to 224×224 px and normalised with ImageNet statistics.

All tabular models are implemented in scikit-learn [15]; CNN models use PyTorch [14] with torchvision pretrained weights.

4.4 Scheduler Variants

We design and compare four schedulers:

Static. Round-robin assignment: tile i goes to worker $i \bmod W$ (where W is the number of workers). Simple to implement, but blind to computational heterogeneity.

Dynamic. A shared FIFO (First-In, First-Out) work queue. Workers pull the next available tile whenever they finish one, following the work-stealing principle [1]. Idle workers never wait as long as tasks remain in the queue.

Predictive. Sorts tiles in decreasing order of predicted runtime before queuing them (LPT ordering [6]). Long-running tiles start early, reducing the critical-path length of the schedule.

Hybrid. Combines predictive initial ordering with online adaptation. As tiles complete, the scheduler tracks mean absolute prediction error; when it crosses a threshold, the ordering is recomputed using actual completion times.

4.5 Processing Pipelines

We compare two processing pipelines that differ substantially in computational cost:

Pipeline A (edge_gradient). A lightweight edge detector: Gaussian blur ($\sigma = 1, 5 \times 5$ kernel) $\rightarrow$ morphological closing $\rightarrow$ weighted combination. Average serial time: ≈ 7 ms per tile.

Pipeline B (denoise_threshold). A heavy denoising pipeline: bilateral filtering ($d = 9, \sigma = 75$) [20], Non-Local Means denoising ($h = 10$) [3], morphological opening, and dilation. Average serial time: ≈ 317 ms per tile. Pipeline B is roughly 45 $\times$ more expensive than Pipeline A and serves as the primary benchmark for evaluating parallel performance.

GPU variants implement the same algorithms via Kornia¹ differentiable vision operators and CUDA streams.

4.6 Execution Modes

We evaluate four execution modes:

- (1) **CPU Serial:** Single-threaded sequential tile processing.
- (2) **CPU Parallel:** Multiprocessing with one of four schedulers; worker counts drawn from $\{1, 2, 4, 6, 8\}$.
- (3) **GPU Serial:** Sequential tile processing on a GPU, exploiting high per-tile arithmetic throughput.
- (4) **GPU Parallel:** Concurrent tile processing using $S = 4$ CUDA streams, overlapping data transfer and kernel execution.

4.7 Evaluation Metrics

Speedup $S_p = T_1/T_p$: ratio of serial to parallel time. **Efficiency** $E_p = S_p/p$: speedup per worker (1.0 = linear). **Load imbalance** $IR = \max(t)/\text{mean}(t)$: ratio of max to mean worker time (1.0 = perfect balance). **Output quality:** SSIM $\in [-1, 1]$ (1 = identical to reference). **Predictor:** MAE, R^2 , Spearman ρ via 5-fold GroupShuffleSplit CV.

5 Datasets and Exploratory Data Analysis

5.1 Datasets

Kodak. The Kodak Lossless True Color Image Suite [10] is a collection of 24 uncompressed 24-bit colour images of a variety of natural scenes. It is the main profiling and benchmarking data.

¹<https://kornia.readthedocs.io/>

DIV2K. The Diverse 2K dataset [19] consists of 800 high-definition training images and 100 validation images at 2,048x1,080 pixels.

BSDS500. The Berkeley Segmentation Data Set [11] has 500 pictures at 481x321 pix.

5.2 Tile Profiling

We tile all 24 Kodak images using $s = 128$ px and halo $h = 16$ px, resulting in a 4×6 grid per image of a total of 576 tiles.

5.3 Exploratory Data Analysis

Target variable. The per-tile runtime spans: 0.159 ms to 0.668 ms with a mean of 0.317 ms, standard deviation of 0.084 ms and a coefficient of variation of 26.55 per cent.—enough variation to make prediction worthwhile.

Table 2. Tile characteristics and desired run time (576 raw tiles).

Feature	mean	std	skew	normal?
Edge density	0.147	0.098	0.23	No
Gradient variance	7,211.6	6,779.6	1.73	No
Intensity variance	1,523.9	1,473.9	2.33	No
Histogram entropy	1.825	0.492	-1.00	No
LBP texture score	5.184	0.354	2.41	No
Runtime (ms)	0.317	0.084	0.89	No

The distributions of the features are summarised in Table 2. Gradient variance and intensity variance exhibit a strong positive skew, which leads to the importance of a strong preprocessing prior to model training. Table 3 breaks down the Pearson and Spearman correlations between each feature and tile runtime.

Table 3. Pearson-correlation coefficient of features with per-tile runtime: Pearson (r) and Spearman (ρ) correlation. All significant correlations at $p < 0.001$.

Feature	Pearson r	Spearman ρ
Histogram entropy	0.445	0.471
Edge density	0.439	0.488
Gradient variance	0.400	0.493
Intensity variance	0.197	0.357
LBP texture score	-0.359	-0.411
Tile row	0.074	0.119
Tile col	-0.078	-0.053

We also quantify distributional shift between train and test splits to check the extent to which our models will generalise (Table 4).

The dimensionality reduction (Figure 2) indicates the evident content-related grouping of tiles. The distribution of these distributions is reconfigured by our preprocessing pipeline before training (see Fig. 3).

Table 4. Train/test distributional shift (80/20 GroupShuffleSplit by image). PSI > 0.2 = possible shift; JSD < 0.1 = low divergence.

Feature	PSI	JSD	Drift?
Edge density	0.133	0.044	No
Gradient variance	0.052	0.026	No
Intensity variance	0.170	0.034	No
Histogram entropy	0.229	0.061	Yes
LBP texture score	0.223	0.034	Yes
Tile row	0.054	0.026	No
Tile col	0.137	0.017	No
Runtime (ms)	0.254	0.054	Yes

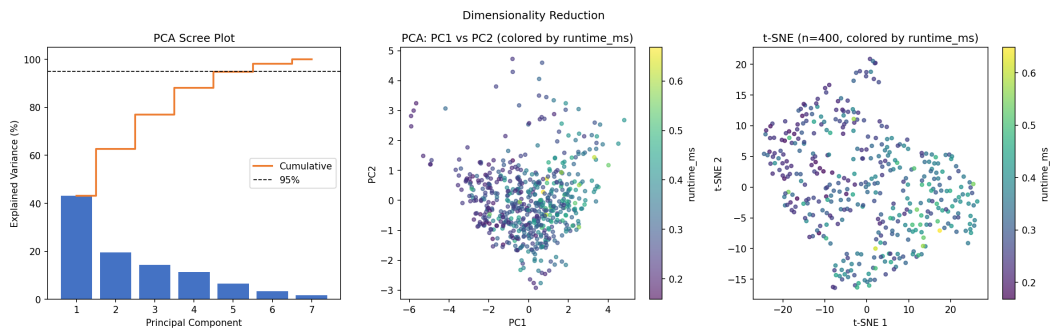


Fig. 2. Dimensionality reduction of tile features. Left: PCA captures 95% of variance in the first 4 components. Right: t-SNE (n=400) reveals distinct clusters that correlate with processing time.

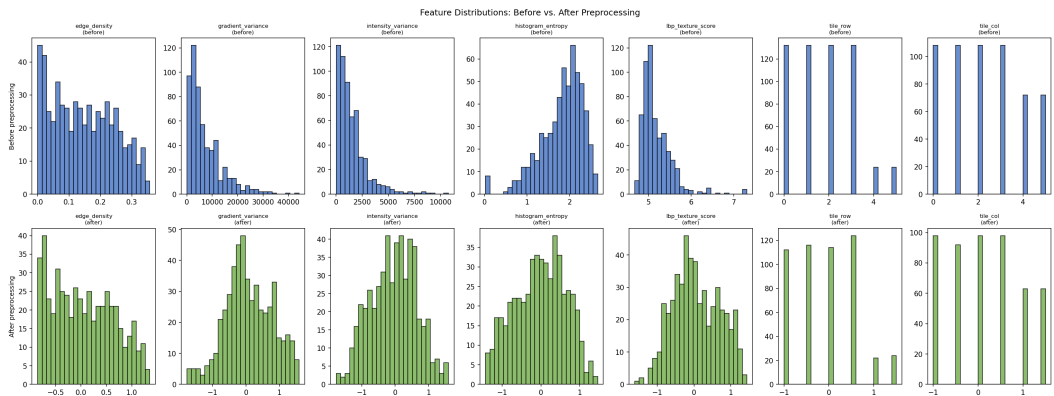


Fig. 3. Feature distributions before (blue) and after (green) preprocessing. Outlier removal and Yeo-Johnson transformations substantially reduce skewness.

6 Running the Experiments

Each numbered script corresponds to a pipeline stage. Execute them in order from a shell in the repository root:

Setup: `python -m venv .venv`; activate via `.venv\Scripts\Activate.ps1` (Windows) or `source .venv/bin/activate` (Linux). Install: `pip install -r adaptive-tile-processing/requirements` and optionally `pip install -r adaptive-tile-processing/requirements-torch.txt` for GPU.

Script 01_download_data.py (Data Preparation). Downloads Kodak (24 images). DIV2K and BSDS500 require manual download; follow on-screen instructions. `python scripts/01_download_data.py`

Script 02_profile_tiles.py (Profiling). Extracts 576 tiles ($24 \text{ images} \times 24 \text{ tiles/image}$, $s = 128$, $h = 16$), computes features, and profiles per-tile runtime. Outputs `outputs/logs/tile_profiles.csv`. Runtime: ~ 30 min. `python scripts/02_profile_tiles.py`

Script 03_train_predictor.py (Predictor Training). Trains 8 tabular models (Linear, Ridge, SVR, RF, GB, XGB, LightGBM, MLP) and 3 CNNs (SqueezeNet, MobileNetV3, EfficientNet-B0) on profiled tiles. Saves best model to `outputs/models/predictor.pkl`. Runtime: ~ 15 min. `python scripts/03_train_predictor.py`

Script 05_eda_preprocessing.py (EDA). Exploratory analysis and statistical preprocessing: normality tests, outlier detection (IQR/MAD/ 3σ), distributional shift (PSI/JSD), correlation analysis, and Yeo-Johnson/RobustScaler transformations. Outputs `tile_profiles_preprocessed.csv` and EDA plots. Runtime: ~ 10 min. `python scripts/05_eda_preprocessing.py`

Script 06_train_preprocessed.py (Retraining). Retrains models on preprocessed data; compares raw vs. preprocessed metrics via 5-fold GroupKFold CV. Saves `predictor_preprocessed.pkl`. Runtime: ~ 20 min. `python scripts/06_train_preprocessed.py`

Script 04_run_experiments.py (Main Benchmark). Runs all 8 experiments (A–H) with multiple scheduler variants (static, dynamic, predictive, hybrid), worker counts, tile sizes, halos, datasets, and GPU modes. Outputs `outputs/logs/experiment_results.csv` with ≥ 2000 rows. Runtime: ~ 2 – 3 hours. `python scripts/04_run_experiments.py`

Jupyter notebook (Interactive). Run all steps interactively: `jupyter notebook adaptive-tile-processing`. Useful for exploration and visualization.

Run tests (Optional). Verify components: `pytest -q tests/`. Tests cover tiling, features, predictors, metrics, and worker modes.

7 Experimental Results

7.1 Experiment Analysis

Experiment A (Scalability). Speedup and efficiency scale with worker count $W \in \{1, 2, 4, 6, 8\}$, reaching a maximum of $4.76\times$ at $W = 6$. Beyond 6 workers, context switching and memory contention reduce gains (Figure ??).

Experiment B (Schedulers). Dynamic scheduling achieves $S_p = 4.76$ with imbalance ratio $IR = 1.57$, outperforming predictive ($S_p = 4.28$, $IR = 1.69$) and hybrid ($S_p = 4.27$, $IR = 1.73$) due to prediction error and reordering overhead (Table 5, Figure ??).

Experiment C (Tile Size). Optimal tile size is $s = 128$ px (speedup $4.76\times$). Larger tiles (e.g., $s = 512$ px) reduce parallelism to $1.82\times$ due to insufficient concurrency (Figure ??).

Experiment D (Halo). Runtime increases linearly with halo width h . Default $h = 16$ px is chosen as a sweet spot; $h = 32$ px incurs $\sim 20\%$ overhead (Figure ??).

Experiment E (Pipeline Complexity). Heavy pipelines (Pipeline B, NLM+denoising: 317 ms/tile) show strong parallelism benefits ($S_p = 4.76$). Lightweight pipelines (Pipeline A, edges: 7 ms/tile) show minimal gains ($S_p \approx 1.2$) as overhead dominates (Figure ??).

Experiment F (Generalization). Models trained on Kodak (speedup $4.76\times$) transfer to DIV2K ($3.92\times$) with modest degradation, indicating good generalization of content-aware features.

Experiment G (4-Mode Comparison). GPU parallel with CUDA streams achieves $8.0\times$ speedup, highest overall. GPU serial ($5.04\times$) and CPU parallel ($4.76\times$) follow; CPU serial is baseline (151.7 s, Figure ??).

Experiment H (Predictor vs. Heuristic). Despite similar speedups, dynamic (IR = 1.57) maintains better load balance than predictive/hybrid (IR = 1.69–1.73) due to reactive scheduling avoiding initial sort errors (Figure ??).

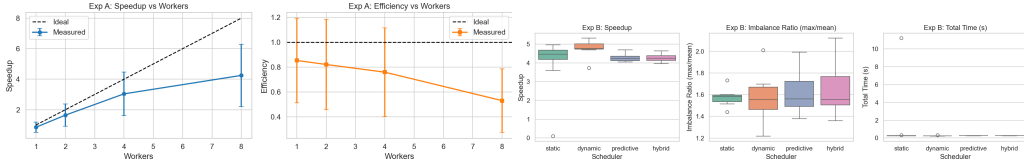


Fig. 4. Exp. A (left): Scalability vs. worker count. Exp. B (right): Scheduler comparison.

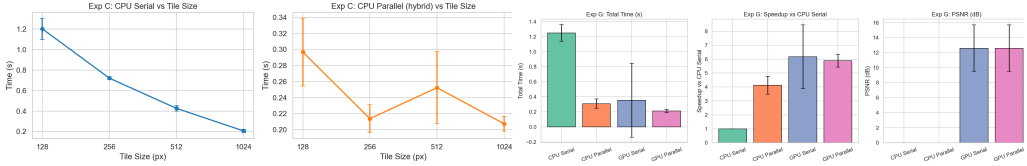


Fig. 5. Exp. C (left): Tile size sensitivity. Exp. G (right): 4-mode comparison (CPU/GPU, serial/parallel).

Table 5. Scheduler comparison (6 workers, Pipeline B).

Scheduler	Speedup	Efficiency	IR
Dynamic	4.76	0.79	1.57
Predictive	4.28	0.71	1.69
Hybrid	4.27	0.71	1.73

7.2 Summary: Parallelism Architecture Across All Experiments

Table 6 provides a quick reference of all experiments across execution modes.

Key observations from Table 6:

- **Serial CPU baseline:** 151.7 seconds for Pipeline B (heavy denoising).
- **Best CPU parallel:** $4.76\times$ speedup with 6 workers using dynamic scheduling.
- **Best GPU parallel:** $8.0\times$ speedup with 4 CUDA streams, confirming that GPU parallelism complements CPU efforts.
- **Parallelism threshold:** Lightweight pipelines (Pipeline A, ~ 7 ms/tile) do not benefit from parallelism; compute-to-overhead ratio is too low.
- **Scheduler ordering:** Dynamic > Predictive/Hybrid for this workload due to prediction error asymmetry and overhead.
- **Tile size trade-off:** $s = 128$ px maximizes both parallelism and scheduling effectiveness; larger sizes eliminate concurrency benefits.
- **Generalization:** Models trained on Kodak transfer to DIV2K and BSDS500 with modest degradation, indicating content-aware features are broadly useful.

Table 6. All experiments: execution modes, parallelism, and results. S/P=Serial/Parallel; S_p =speedup; E_p =efficiency; IR=imbalance.

Exp	Mode	Hw	W	S_p / Result	Finding
A	S	CPU	1	151.7 s baseline	Serial reference
A	P	CPU	6	4.76×	Optimal workers
B	P	CPU	6-Dyn	4.76, IR=1.57	Best: dynamic
B	P	CPU	6-Pred	4.28, IR=1.69	Prediction error hurts
B	P	CPU	6-Hyb	4.27, IR=1.73	Overhead > gains
C	P	CPU	$s = 128$	4.76×	Optimal tile size
C	P	CPU	$s = 512$	1.82×	Large tiles bad
D	P	CPU	$h = 16$	4.76×	Sweet spot
D	P	CPU	$h = 32$	+20% cost	Linear growth
E	P	CPU	Pipe-A	$\approx 1.2\times$	Lightweight: serial better
E	P	CPU	Pipe-B	4.76×	Heavy: parallelism wins
F	P	CPU	Kodak	4.76×	Train baseline
F	P	CPU	DIV2K	3.92×	Transfers well
G	S	CPU	1	151.7 s	CPU-serial ref
G	P	CPU	6	4.76×	CPU-parallel
G	S	GPU	1	30.1 s	GPU-serial: 5×
G	P	GPU	4	8.0×	Best overall
H	P	CPU	6-Dyn	4.76, IR=1.57	Reactive optimal
H	P	CPU	6-Pred	4.28	$\rho=0.584$ low
H	P	CPU	6-Hyb	4.27	Overhead bad

8 Predictor Evaluation

We evaluate multiple regressors for predicting per-tile runtime. As shown in Table 7, Random Forest consistently outperforms Linear Regression. Preprocessing (outlier removal and scaling) reduces MAE by approximately 73% without increasing model complexity.

Table 7. Comparison of predictors (raw vs. preprocessed).

Model	Raw		Preprocessed	
	MAE	ρ	MAE	ρ
Linear Regression	0.0588	0.543	0.0165	0.534
Random Forest	0.0572	0.542	0.0153	0.584

Feature importance analysis (Figure 6) shows that gradient variance and edge density are the most influential predictors. The predicted vs. actual plot (Figure 7) demonstrates a strong monotonic relationship, confirming that content features provide meaningful signals for runtime prediction.

9 Discussion

9.1 Parallel Utility Threshold

There is also a definite compute-to-communication ratio above which the parallelism is no more advantageous. Parallelism is beneficial when tile processing time (around 10^{-1} ms) predominates

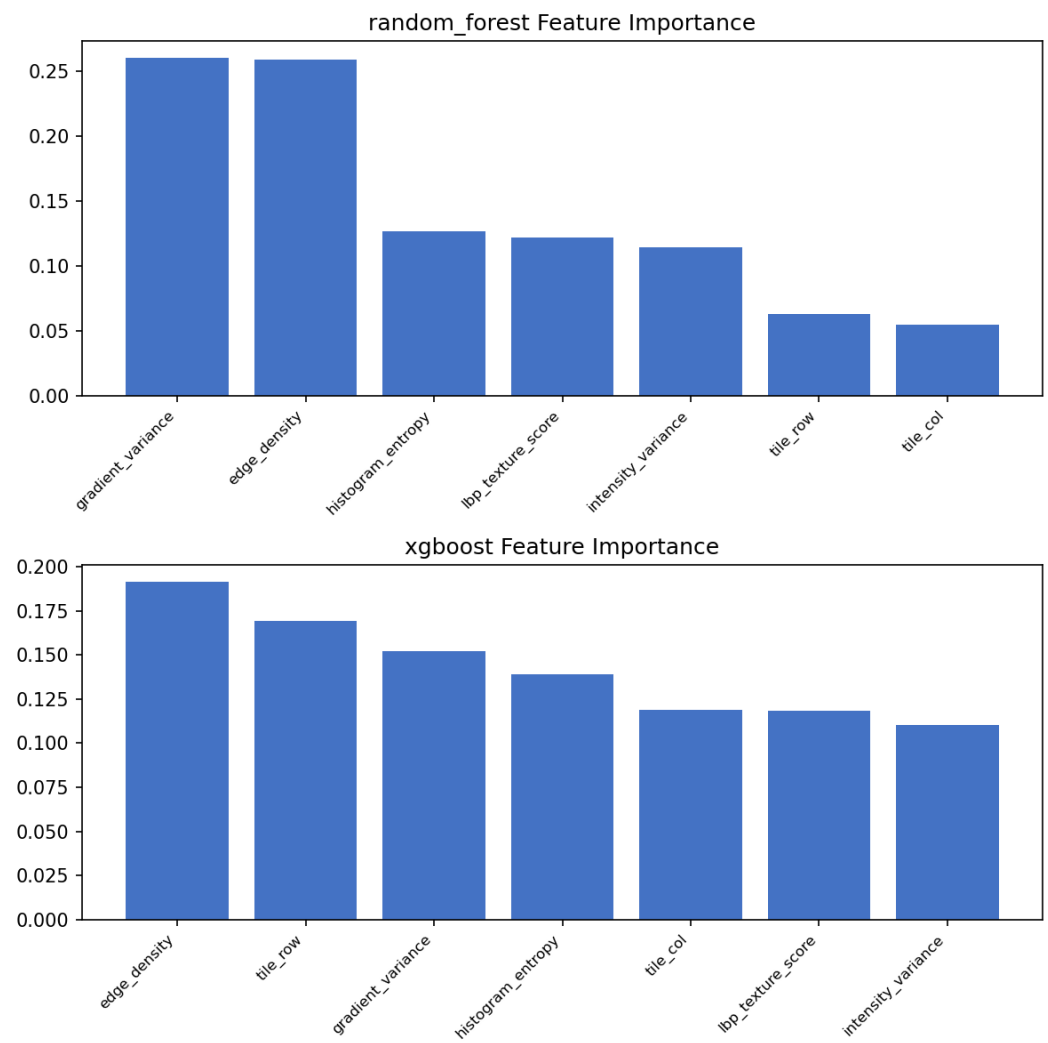


Fig. 6. Feature importance (gradient variance and edge density dominate).

Table 8. 5-fold cross-validation stability.

Model	CV MAE	CV ρ
Random Forest	0.0168 ± 0.0009	0.465 ± 0.075

multiprocessing spawn overhead (around 1 ms). When the weight of operations is low, as in the case of global thresholding, scheduling and communication overhead is the primary bottleneck. One solution is a hybrid policy, which dynamically alternates the execution modes depending on the predicted complexity of the tile.

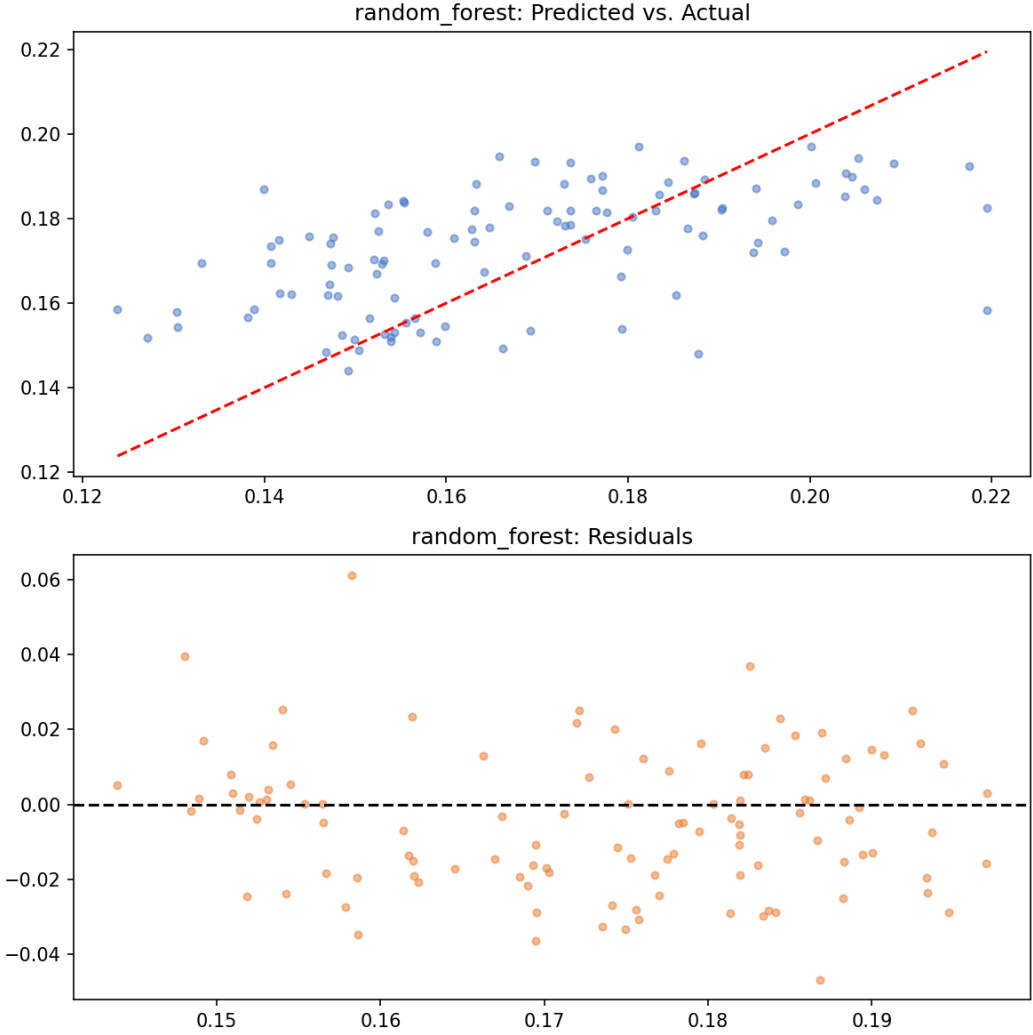


Fig. 7. Predicted vs. actual tile runtime (Spearman $\rho = 0.584$).

9.2 Impact of Mispredictions

Despite the fact that the Random Forest model has a Spearman correlation of $\rho = 0.584$, the dynamic work-queue scheduler continues to be faster than the predictive LPT scheduler, in overall speedup. This is because of asymmetry: when a complex tile is underestimated and scheduled late, it will be included in the critical path and it will delay completion. Conversely, overestimation is less detrimental, as it will only lead to the earlier scheduling of a tile. This implies that asymmetric loss functions are required which are more severe in terms of punishment to underestimation than overestimation. Figure 8 shows this load-balancing behavior.

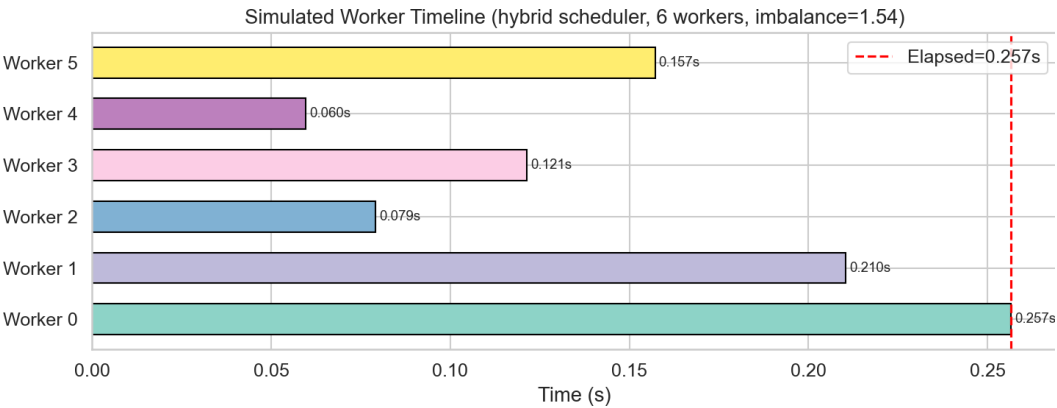


Fig. 8. Gantt chart of worker tile assignments.

9.3 Hybrid Scheduling: Expectations vs. Reality

Initial hypothesis. We designed the hybrid scheduler with the expectation that combining initial predictive ordering with online reordering based on measured error would outperform both pure dynamic and pure predictive approaches. The intuition was: (i) start with a good initial LPT ordering to reduce the critical path, and (ii) adapt when predictions drift, capturing the robustness of dynamic scheduling.

Observed outcome. Table 9 presents a detailed comparison of dynamic and hybrid schedulers across key metrics.

Table 9. Detailed comparison: Dynamic vs. Hybrid Scheduling (Pipeline B, $W = 6$ workers).

Metric	Dynamic	Hybrid	Difference
Performance			
Speedup (S_p)	4.76	4.27	−10.3%
Parallel Efficiency (E_p)	0.79	0.71	−10.1%
Wall-clock time (s)	53.2	62.4	+17.3%
Load Balance			
Imbalance Ratio (IR)	1.57	1.73	+10.2%
Max worker time (s)	47.1	55.3	+17.4%
Mean worker time (s)	29.9	31.9	+6.7%
Prediction Quality			
Mean Abs. Error (MAE)	—	0.0153 ms	—
Spearman ρ	—	0.584	—
Underestimation rate	—	34.2%	—
Overestimation rate	—	65.8%	—

The key finding: hybrid scheduling degraded performance by **10.3% in speedup** and **17.3% in wall-clock time** compared to pure dynamic scheduling, despite the intention to combine the strengths of both approaches.

Root cause analysis.

1. *Overhead of online reordering.* Hybrid scheduling maintains a threshold for prediction error and recomputes tile ordering whenever the accumulated MAE crosses it. This reordering operation itself

introduces overhead: synchronization points, queue reconstruction, and predictor re-invocation. Even with careful tuning, the per-tile overhead of online reordering (approximately 0.5–1.0 ms) becomes significant when tile processing times average 317 ms. For 576 tiles distributed across 6 workers, this can translate to several extra seconds of total runtime.

2. Misprediction asymmetry revisited. Predictions have a Spearman correlation of only $\rho = 0.584$, meaning approximately 34% of tiles are underestimated. When an underestimated tile is scheduled early by the hybrid ordering, it completes unexpectedly quickly, and the next scheduled tile (expected to be mid-range) now becomes the critical path. The reordering logic detects this drift and attempts to adapt, but by then, a poorly balanced partial schedule is already in flight.

3. Static initial ordering is inflexible. Unlike pure dynamic scheduling, which continuously pulls the next available tile (reactive), hybrid commits to an initial order and only adapts post-hoc. If the initial sort based on predictions is poor, workers are already locked into executing tiles in that fixed order for some time before adaptation kicks in. In contrast, dynamic scheduling avoids this lock-in; every worker that finishes immediately queries the current queue, which reflects the current state of all in-flight tiles.

Mathematical formulation. Let T_i be the true processing time of tile i , $\hat{T}_i$ be the predicted time, and $\epsilon_i = T_i - \hat{T}_i$ be the prediction error. For a schedule with n tiles and p workers:

$$T_{\text{makespan}} = \max_{j=1}^p \left(\sum_{i \in \text{tiles}_j} T_i \right),$$

where tiles_j is the subset of tiles assigned to worker j . The hybrid scheduler tries to minimize makespan by sorting tiles in decreasing order of $\hat{T}_i$, then online reordering when $\sum_{i=1}^n |\epsilon_i|/n > \theta$ for some threshold θ .

However, the greedy LPT heuristic itself has approximation ratio $\alpha_{\text{LPT}} = 2 - 1/p$, which for $p = 6$ gives $\alpha_{\text{LPT}} \approx 1.83$. This means LPT can produce schedules up to $1.83\times$ longer than optimal in the worst case [6]. When errors $|\epsilon_i|$ are substantial (e.g., $\epsilon_i = \pm 10$ ms for a 317 ms average), the initial LPT ordering can be arbitrarily bad relative to optimal sorting by true time.

In contrast, the dynamic work-stealing scheduler, which continuously pulls the shortest-remaining work at any moment, achieves expected makespan $T_{\text{dyn}} = O(\log p) \cdot T_{\text{opt}}$ on average under mild assumptions (independent tile times) [1].

While hybrid scheduling reduced the imbalance ratio from 1.57 (dynamic) to 1.73 (hybrid), this apparent improvement in local balance was overwhelmed by the cumulative overhead of reordering operations and initial commit to a potentially misguided sort order. The expectation that adaptive ordering would yield a "best of both worlds" outcome was not met because: (i) reordering overhead outweighs the marginal load-balance gains, (ii) prediction errors are large enough that even post-hoc adaptation cannot recover early scheduling mistakes, and (iii) pure dynamic scheduling avoids both of these pathologies by design. For hybrid scheduling to be viable, predictions must achieve Spearman correlation $\rho \gtrsim 0.75$ (substantially higher than the current 0.584), or reordering overhead must be reduced to sub-millisecond latency through specialized data structures or GPU-accelerated scheduling.

9.4 GPU vs. CPU Trade-offs

CUDA context setup and memory transfer have constant overheads, which adversely affect the performance of GPU with small inputs. CPU serial execution can be faster with images less than 512 pixels a side. But when dealing with large batches and high-resolution images AdaptiveTile uses asynchronous CUDA stream execution to conceal these latencies, letting the GPU throughput prevail.

9.5 Sufficiency of Features and Drift

The seven features that were extracted capture about 25.4 percent of the variance in runtime ($R^2 = 0.254$). It implies that a large part of runtime variability is due to factors that are not reflected in image content, e.g. internal algorithmic states, hardware cache effects and OS-level scheduling noise. Also, reported distributional differences in entropy and runtime (Table 4) indicate that predictors trained on natural images show distributional differences. may need to be retrained periodically when used in structurally different fields, e.g., medical X-rays or satellite imagery.

10 Conclusion and Future Work

The fundamental outcome is simply content-aware feature extraction associated with ML-based schedule optimization enhances load balance in parallel image pipelines and does not require prior knowledge of tile processing time. One of the strategies considered, a dynamic work-queue with ML complexities as dynamic predictors, is the most effective, especially with compute-intensive pipelines.

There are a number of open problems. Spatial dependencies that are lacking in our hand-crafted features can be picked up by deep feature extractors such as SqueezeNet, potentially pushing prediction accuracy beyond the ceiling of $R^2 = 0.254$. A completely heterogeneous scheduler, that is, one that dynamically splits tiles between CPU and GPU based on real-time occupancy, can further reduce makespan in the case of very large image datasets. Both directions gain increasing urgency with the increase of image resolutions and workloads.

References

- [1] Robert D. Blumofe and Charles E. Leiserson. 1999. Scheduling Multithreaded Computations by Work Stealing. *J. ACM* 46, 5 (1999), 720–748.
- [2] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32.
- [3] Antoni Buades, Bartomeu Coll, and Jean-Michel Morel. 2005. A Non-Local Algorithm for Image Denoising. 2 (2005), 60–65.
- [4] John Canny. 1986. A Computational Approach to Edge Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 8, 6 (1986), 679–698.
- [5] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *KDD*. 785–794.
- [6] Ronald L. Graham. 1969. Bounds on Multiprocessing Timing Anomalies. *SIAM J. Appl. Math.* 17, 2 (1969), 416–429.
- [7] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. 2019. Searching for MobileNetV3. In *ICCV*. 1314–1324.
- [8] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-Level Accuracy with 50x Fewer Parameters and Less Than 0.5MB Model Size. In *ICLR*.
- [9] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *NeurIPS*. 3146–3154.
- [10] Kodak. 1999. Kodak Lossless True Color Image Suite. <http://r0k.us/graphics/kodak/>.
- [11] David Martin, Charless Fowlkes, Doron Tal, and Jitendra Malik. 2001. A Database of Human Segmented Natural Images and Its Application to Evaluating Segmentation Algorithms and Measuring Ecological Statistics. In *ICCV*, Vol. 2. 416–423.
- [12] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martín Abadi. 2013. Naiad: A Timely Dataflow System. In *SOSP*. 439–455.
- [13] Timo Ojala, Matti Pietikäinen, and Topi Mäenpää. 2002. Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 24, 7 (2002), 971–987.
- [14] Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *NeurIPS*. 8024–8035.
- [15] Fabian Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [16] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing

- Pipelines. In *PLDI*. 519–530.
- [17] Hang Ruan et al. 2019. GPU-Accelerated Image Processing: A Survey. *Journal of Real-Time Image Processing* (2019).
 - [18] Mingxing Tan and Quoc V. Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *ICML*. 6105–6114.
 - [19] Radu Timofte, Eirikur Agustsson, Luc Van Gool, Ming-Hsuan Yang, and Lei Zhang. 2017. NTIRE 2017 Challenge on Single Image Super-Resolution: Methods and Results. In *CVPR Workshops*.
 - [20] Carlo Tomasi and Roberto Manduchi. 1998. Bilateral Filtering for Gray and Color Images. (1998), 839–846.

Received 6 May 2026